DocuSign API + Workflows on Java Spring Boot

Printable Interview Cheat Sheet

Use this sheet for a 30-minute interview. Keep answers brief, architecture-focused, and production-oriented.

1) 30-Second Summary Answer

Best opener: “In Spring Boot, I integrate DocuSign through the Java SDK, use **Authorization Code Grant** for interactive user flows and **JWT Grant** for server-to-server automation, send envelopes using **templates or composite templates**, support **embedded signing** with recipient view when users are already in the portal, and track state with **DocuSign Connect webhooks** secured by **HMAC**. If the process needs orchestration beyond signing, I use **Workflow Builder API**.”

2) Architecture You Can Describe

- **Auth service**: obtains and refreshes OAuth token; injects token into ApiClient.
- **Envelope service**: creates/sends envelopes, templates, recipients, tabs.
- **Signing service**: generates recipient view URL for embedded signing.
- **Webhook controller**: receives Connect events, validates HMAC, stores payload, processes async.
- **Workflow service**: triggers and monitors Workflow Builder instances when the business process has multiple steps.
- **Persistence**: stores envelopeId, recipient status, event history, correlation IDs, timestamps.

3) OAuth Decision Rule

- **Authorization Code Grant** → user-interactive web apps where the user signs in and consents.
- **JWT Grant** → backend automation, service-to-service jobs, scheduled processes.
- Always request the **signature** scope for eSignature API calls.
- Every eSignature API call requires a valid **access token**.

4) Sending Envelopes

- Create an **EnvelopeDefinition**.
- Add documents directly or reference one or more **templates**.
- Add recipients and tabs; set status to **sent** to send immediately.
- Use **templates** for repeatable/static packages.
- Use **composite templates** when combining server templates with dynamic documents or recipient data.

5) Tabs / Field Placement

- **Fixed tabs**: use X/Y coordinates when the PDF layout is stable.
- **Anchor tabs** (AutoPlace): use anchor text when layout can shift.
- Anchor tabs are better for dynamic/generated PDFs and long-term maintainability.
- If possible, use short and unique anchor strings for performance and accuracy.

6) Embedded vs Remote Signing

- **Remote signing**: recipient gets email/SMS/WhatsApp from DocuSign.
- **Embedded signing**: your app requests a **recipient view URL** and hosts the signing flow inside your app/portal.
- For embedded signing, the signer must be identified with a **clientUserId**.
- Recipient view URL is **single-use** and expires quickly—generate it just-in-time and never store it.

7) Webhooks / Connect

- Prefer **DocuSign Connect** over polling.
- Common events: **envelope-sent**, **envelope-delivered**, **envelope-completed**, **envelope-declined**, **envelope-voided**.
- Webhook listener should: validate → persist → acknowledge fast → process async.
- Make processing **idempotent** so retries or duplicates do not trigger duplicate business actions.

8) HMAC Validation Talking Points

- Enable HMAC in Connect configuration.
- Read the **raw request body exactly as sent** (including line endings).
- Compute **HMAC-SHA256** with the shared secret; base64-encode it.
- Compare against **X-Docusign-Signature-1** (or other configured signature headers).
- Reject the request if no computed signature matches.

9) Workflow Builder API — When to Mention It

- Use it when the process is broader than a single envelope lifecycle.
- Good for multi-step agreement flows: forms → signatures → approvals → external actions.
- Your app can **trigger workflows** and **manage workflow instances** (monitor, pause, resume, cancel).

10) High-Value Interview Q&A; Nuggets

- **Why templates?** Better reuse, consistency, and lower maintenance.
- **Why composite templates?** More flexible than a single template; combine templates with runtime documents/data.
- **Why embedded signing?** Best UX when users are already signed into the app.
- **Why webhooks not polling?** Lower latency, lower load, event-driven architecture.
- **What is production-ready?** Idempotency, async processing, HMAC validation, audit logging, externalized config.

11) Keywords to Say Out Loud

Authorization Code Grant • JWT Grant • signature scope • EnvelopeDefinition • Envelopes API • Templates • Composite Templates • Tabs • Anchor tabs • createRecipientView • clientUserId • returnUrl • Focused View • Connect • HMAC • idempotency • webhook listener • Workflow Builder API

12) Red Flags to Avoid in Answers

- Do **not** say you store the embedded signing URL.
- Do **not** say polling is preferred over Connect.
- Do **not** put heavy business processing directly inside the webhook request thread.
- Do **not** hardcode secrets, integration keys, or private keys in source code.
- Do **not** assume fixed X/Y tabs are always reliable for generated documents.

13) Fast Answers (1 sentence each)

- **What is clientUserId?** It uniquely marks an embedded signer in recipient view requests.
- **Why HMAC?** It verifies webhook authenticity and payload integrity.
- **Why async webhook processing?** To acknowledge fast and avoid retries/timeouts.
- **When use JWT?** For server-to-server integration without interactive login.
- **When use Workflow Builder?** When agreement steps span more than envelope sending/signing.

14) Mini End-to-End Example

User creates contract in your portal → Spring Boot service authenticates with DocuSign → app builds envelope from template/composite template → recipient signs remotely or through embedded signing → Connect posts status changes to your webhook → webhook validates HMAC, stores event, updates contract status, and triggers downstream provisioning/archiving.

> Tip: In interviews, always connect DocuSign features to software engineering principles: **security**, **UX**, **maintainability**, **resilience**, and **event-driven design**.

Prepared as a concise study sheet for interview prep.